

Exercise 3: Arrays and Matrix Algebra

Note: Before you start this exercise, if you already have Octave running and some variables defined, remove them with the “clear” command. Make a folder for today’s work in your Octave working directory, with a dated label in YYMMDD format. Remember to make notes in your research notebook as well!

1. Arrays

Sometimes we want to store a large number of related variables during our programs. For example, what if we wanted to specify the position of a particle in three dimensional space? One way to do this would be to use three different variables to save the coordinates along the x, y, and z axes of our 3D space. This might look like:

```
position_x = 2;  
position_y = 3;  
position_z = 5;
```

This sequence of commands will work, but it is a bit clunky. It takes a lot of lines, and we are representing a single quantity (position in space) with three different variables. To do this a bit better, we can use an array, which is a variable that contains multiple values. To do this, would could use a command like:

```
position_xyz = [2 3 5]
```

Type (or paste) this command into your Command Window and press enter. Octave should create a variable called position_xyz, and assign it three values. The three values are called elements of the array. They are just like the three variables with different names that we defined above, except that because they are all part of one array we will be able to instruct the computer to work on them as a group. Because we left off the usual line-ending semicolon, Octave will also report back the value of our variable. This output should be in your Command Window:

```
position_xyz =  
      2      3      5
```

This output shows that the three values are contained as what we call a row-vector. You can also see this information in your Workspace, which should specify that the variable position_xyz has

Dimensions 1x3. This means the array `position_xyz` has one row and three columns. We can change `position_xyz` to a column vector by putting a semicolon between the values:

```
position_xyz = [2; 3; 5]
```

If you run this command you will see the output:

```
position_xyz =  
  
2  
3  
5
```

You will also notice that in your Workspace the variable `position_xyz` has Dimensions 3x1, three rows and one column. By using semicolons we redefined the variable as a column vector, and that is reflected in the displayed output.

The advantage of linking our three variables together as an array is that we can now make changes to all three at once. For example if we want to add one to each element of the array, we can write:

```
position_xyz = position_xyz + 1;
```

If you run this command you will see that it increases the value of each element of the array by one. We can also subtract, multiply, divide, etc.

If we don't want to perform an operation on all the elements of our array, we don't have to. We can specify any given element of our array using parentheses with a number inside. So for example, to refer to y-axis position of our particle we can write:

```
position_xyz(2)
```

If we type this in the command line, we should get back the value stored in the second element of our array, currently 4. Previously we called our array a row- or column-vector. Vector is math terminology for a one dimensional array. What we mean by one dimensional is that we only need one number inside the parentheses to specify an element of the array.

We can have two dimensional arrays as well. Let's say we add a second particle to our 3D space. It makes sense to keep the coordinates of both particles in the same array, in case we want

to do anything to the whole system. Create a script to set up this system using the script Editor, using this template:

```
%file created by XXX on YYMMDD
%tutorial exercise 3
clear
position_xyz(1,1:3) = [2 3 5];
position_xyz(2,1:3) = [7 11 13];
```

This script will create a two dimensional array that stores the x,y,z coordinates of particle 1 as the first row (values 2, 3, and 5) and the coordinates of particle 2 as the second row (values 7, 11, and 13). The math term for a two dimensional array is a matrix.

Let's think about how Octave interprets the last two lines of this script. When Octave sees line 4 of the program, it first checks to see if `position_xyz` exists already. Since we just cleared all the variables in line 3, it will have to create this variable. The `(1,1:3)` part of line 4 tells Octave that we want to work with the elements in columns one through three of the first row of the `position_xyz` array. Again, if these do not already exist, Octave will create them. The `=` symbol tells Octave that we will be assigning values to this element. Octave is now expecting to be fed three values as a row-vector. We provide those values as `[2 3 5]`. *[Note: If we mess up and accidentally give Octave a column vector here by typing `[2; 3; 5]`, Octave will usually figure out what we meant and make the assignment we wanted. This only works because our program is simple, in general we need to be careful about when we mean to refer to a row and when we want to aim for a column].* Finally, the semicolon tells Octave that we are finished with this line of the program, and the Command Window output should be suppressed. Line 5 of our program follows analogous logic to assign values to the second row of our array.

To specify a single element of `position_xyz`, we now need to put two numbers into parentheses. If we want to specify the x-axis position of particle 2, we would need to write:

```
position_xyz(2,1)
```

If we type this in the Command Window, we should get back the value stored in the second row, first column of our array, which is 7.

If we think about the 2D array we just made, we can see that it has a lot of information in it. Each row refers to a particle, and each column refers to an axis of our 3D space. We can imagine expanding our array to hold information about more particles (adding rows), or more properties of each particle (adding columns). Giving our data this kind of structure makes it easy for us to think about and manipulate. It is very important to keep in mind that Octave doesn't know or care about any of this structure. If we tell it to assign z-axis positions to the first column, it will do so happily. All computers do is move numbers around. They are very quick about it, but they can't think about what the numbers mean. It is our job as scientists/programmers to keep track of the structure of our data and make sure we get physically meaningful results.

2. Matrix Algebra

In our scientific programs, we will frequently be doing mathematical operations on vectors and matrices. Octave will take care of the dirty work, but we need to understand what it is doing. A great way to build this understanding while getting some Octave practice is to build scripts that verify how each operation works.

2.1 Addition and Subtraction

We can add two matrices together only if they have the same number of rows and columns. To do this, we simply add the elements of each matrix that have the same position.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

Octave knows which of our variables are matrices, so if we want to add two matrices together we can just use a simple addition sign. Subtraction works in an analogous fashion. To test all this out, we can use this script below. First we create two matrices. Next we add them together with a single line and save the result to a new variable. Then we create another variable, and we write out the element-by-element addition to carry out the matrix addition. Now we can verify that both methods (line 9, and lines 11–14) give the same result. Vector addition works the same way. In this case we can treat vectors as a special case of matrices with only one element in the row or column dimension. *[Note: Octave will still try to carry out our addition if we dimensions of our two matrices are mismatched, expanding our two matrices as needed and treating the missing elements as zeros.]*

```
%file created by XXX on YYMMDD  
%tutorial exercise Matrix Algebra
```

```
clear
```

```
matrix1 = [2 3; 5 7];  
matrix2 = [11 13; 17 19];
```

```
matrix_added_A = matrix1 + matrix2;
```

```
matrix_added_B(1,1) = matrix1(1,1) + matrix2(1,1);  
matrix_added_B(1,2) = matrix1(1,2) + matrix2(1,2);  
matrix_added_B(2,1) = matrix1(2,1) + matrix2(2,1);  
matrix_added_B(2,2) = matrix1(2,2) + matrix2(2,2);
```

```
matrix_added_A  
matrix_added_B
```

2.2 Matrix Multiplication

There are a few different types of multiplication we have to be familiar with. If we multiply a matrix by a single number, a.k.a. a “scalar value”, then we just multiply each element of the matrix by that number. This is called scalar multiplication.

$$c \cdot \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} c \cdot a_{11} & c \cdot a_{12} \\ c \cdot a_{21} & c \cdot a_{22} \end{bmatrix}$$

To carry out scalar multiplication in Octave we can just multiply our 2D array variable by a number, and Octave will know what we mean.

```
product_variable = scalar_value * matrix_variable;
```

If we multiply two matrices, they are required to have the property that the number of rows in the first matrix be equal to the number of columns in the second matrix, and vice-versa. For example, we can multiply a 5x2 matrix with a 2x5 matrix. We can’t multiply a 5x2 matrix with a 3x4 matrix. Each element of the produce matrix is calculated by taking the corresponding row from the first matrix and the corresponding column from the second matrix, then multiplying their elements sequentially and summing the result, as shown below.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \bullet \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} & a_{11}b_{13} + a_{12}b_{23} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} & a_{21}b_{13} + a_{22}b_{23} \\ a_{31}b_{11} + a_{32}b_{21} & a_{31}b_{12} + a_{32}b_{22} & a_{31}b_{13} + a_{32}b_{23} \end{bmatrix}$$

The Octave command for multiplying two matrices is much simpler, we just multiply the two array variables. *[Note: If the dimensions don't match up properly, Octave will let us know and will not run our program.]*

```
product_variable = matrix_variable1 * matrix_variable2;
```

If we have one row vector and one column vector, and they each have the same number of elements, we can calculate what is called a “scalar product” or “dot product”. The method is identical to matrix multiplication, and we get out a single number at the end.

$$\begin{bmatrix} a_{11} & a_{12} \end{bmatrix} \bullet \begin{bmatrix} b_{11} \\ b_{21} \end{bmatrix} = a_{11}b_{11} + a_{12}b_{21}$$

In Octave we can do this in two different ways. We can simply multiply the two vector variables.

```
product_variable = vector_variable1 * vector_variable2;
```

Alternatively, we can use the dot product function.

```
product_variable = dot(vector_variable1,vector_variable2);
```

Vectors in 3D space, such as the position vectors defined in the first part of this exercise, also have some interesting properties. We'll get into this later. We also will leave matrix division aside for now, as it requires some further calculations that we don't want to deal with yet.

2.3 Element-by-Element Math

It's important to remember that our data structure, the array, is different from a mathematically defined vector or matrix. We don't always have to follow the rules of mathematics for matrices when we handle a 2D array. We only use the matrix rules when we want to treat the data stored in the array as a matrix. For example, let's say we are managing a warehouse and we want to write some software for working with our inventory. We might store the number of each type of item that we have in stock in a vector. In another vector, we might store the sale price of each item. If we are smart, we will make the order of the items consistent,

so that the first elements of each vector corresponds to the same item, and the second elements to the next item, etc. If we are really smart, we can even store these two vectors as columns (or rows) in the same matrix. If we want to calculate the total amount of money we would get from selling all items, that would just be the dot product of our two vectors.

$$total_capital = price_vector * stock_vector;$$

If we want to know the amount we could get from selling out of each individual item, we want to multiply each element of our price vector by the corresponding element of our stock vector. We can instruct Octave to do this by putting a “.” in front of our multiplication sign.

$$capital_vector = price_vector .* stock_vector;$$

Of course, we could also have done this with loops. Using the “.*” command has the advantage of being much faster and simpler to code into our program, making it easier to read and less prone to error. In a lot of situations, the computer is also able to execute this command faster.

Octave supports analogous commands for division (./), raising numbers to powers (.^), etc. We should use operations instead of writing out loops any time we can.

3. Practice

1. Go back to the table of Hydrogen energy levels from Exercise 2, and rewrite your program so that it stores all energy level names and energy values in an array, then expresses that array as a table in the Command Window.
2. Write a program that calculates the various types of matrix and vector algebra element by element, and verifies that the simpler, one-line octave commands give the same result. Use the vector addition code provided as a template. You may have noticed that the matrices in the template code only contain prime numbers. This helps make your any bugs in your code easily traceable, since each prime number has a unique effect during mathematical operations. Do not move on until you can write and understand programs that verify these math tools.
3. Write a program that compares the results of the “normal” version of standard mathematical operations (+, -, *, /, and ^) with the “element-by-element” versions (.+, .-, .*, ./, and .^). Do not move on until you can write and understand programs that verify these math tools.
4. Write an example of the inventory program described in section 2.3. Store the data in the table below in a matrix, then calculate the total sale value of the inventory and the total profit possible from the sale of all inventory.

Item	Cost to Make (\$)	Sale Price (\$)	# in Stock
A	0.12	0.45	452
B	2.11	8.99	9846
C	1.96	13.99	354
D	0.42	1.99	6512
E	0.88	1.15	987